FLUTTER LAB 3B

Travel App

Multi-Screen Flutter UI Design & Navigation

Course	Mobile Application Development
Lab	Lab 3 – Flutter Design and Navigation
App Name	Travel App
Platform	Flutter / Dart (Material 3)
Date	March 2026

Done By:
Patrick Straton MBABAZI and
Vincent Habayimana

Lab Report submitted as part of the Flutter UI & Navigation assignment. All data is hard-coded inside Dart source files. No external APIs or databases are used.

1. App Overview

Wanderlust is a hard-coded, multi-screen travel discovery application built entirely with Flutter and Dart. The app allows users to browse curated travel destinations, view detailed information, and simulate a booking flow — all without connecting to any backend or external data source.

The project demonstrates proficiency in advanced Flutter UI composition, custom widget design, navigation between screens using Navigator.push/pop, consistent theming with Material 3, and state management with StatefulWidget.

1.1 Project Structure

The codebase is organized into clearly separated concerns:

- lib/main.dart — App entry point, MaterialApp configuration and global theme
- lib/data/travel_data.dart — Hard-coded destination data (Destination model + TravelData class)
- lib/Screens/home_screen.dart — Home Screen with search, category filter, and destination grid
- lib/Screens/detail_screen.dart — Detail Screen with full destination information
- lib/Screens/booking_screen.dart — Booking Screen with form, price summary, and confirmation dialog
- lib/widgets/destination_card.dart — Reusable DestinationCard widget
- lib/widgets/info_chip.dart — Reusable InfoChip and HighlightTile widgets
- assets/images/ — Local image assets for all 8 destinations

1.2 Color Theme

The app uses a consistent color scheme throughout all screens:

Primary Indigo	#6366F1 — Buttons, active states, labels
Dark Navy	#1A1A2E — Headings, primary text
Background Gray	#F8F9FA — Scaffold background color
Category Colors	Beach: Cyan · Mountain: Green · City: Purple · Adventure: Orange · Cultural: Pink

2. Screen-by-Screen Breakdown

2.1 Home Screen

The Home Screen acts as the main dashboard of the app. It is implemented as a StatefulWidget to support interactive category filtering and live search functionality.

Layout Structure

The screen uses CustomScrollView with multiple SliverToBoxAdapter sections to achieve a smooth, unified scroll experience combining the header, search bar, category chips, and the destination grid.

Key Components

- AppBar area — Custom header with "Discover the World" title, notification icon button, and a gradient avatar (initials "IJ")
- Search Bar — TextField inside a white rounded Container with prefix search icon and a styled filter button; search is live and filters the grid in real time
- Category Chips — Horizontal ListView.builder with 6 categories (All, Beach, Mountain, City, Adventure, Cultural), each with a matching icon
- Destination Grid — SliverGrid with 2-column layout using childAspectRatio: 0.65; powered by DestinationCard widgets
- Favorite Toggle — Each card has an interactive heart icon button with local state

Screenshots

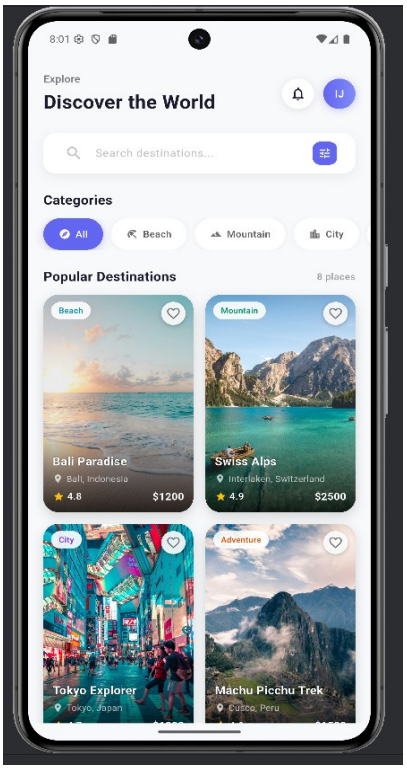

Figure 1 – Home Screen (default view)

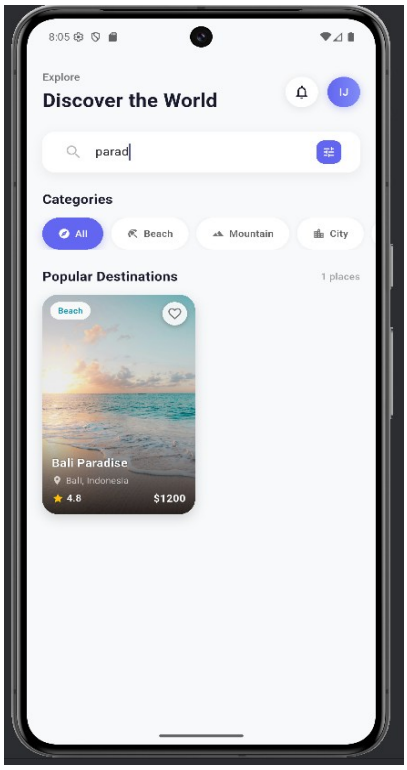

Figure 2 – Search functionality

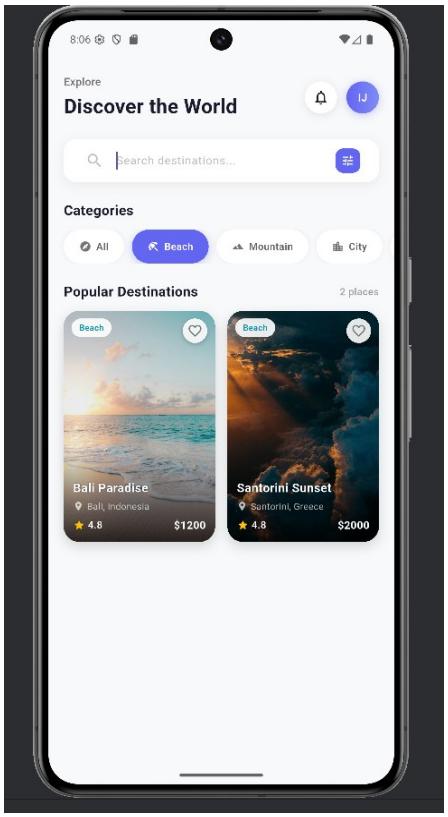

Figure 3 – Category filter active (Beach)

2.2 Detail Screen

The Detail Screen presents comprehensive information about a selected destination. It receives a Destination object via its constructor, passed from the Home Screen through Navigator.push.

Layout Structure

The screen uses a Stack at the root level containing a SingleChildScrollView for all scrollable content, and a Positioned bottom bar for the "Book Now" button. This allows the price and CTA button to always remain visible regardless of scroll position.

Key Components

- Header Image — Full-width Image.asset at 340 px height with BoxFit.cover, overlaid with back and share circle buttons
- Curved Transition — A white Container with borderRadius at the bottom of the image creates a smooth curve into the content area
- Title & Rating — Row containing the destination name, location (with pin icon), and an amber star rating badge
- InfoChips — A Wrap of three InfoChip widgets showing duration, category, and price
- About Section — Full scrollable description text with 1.6 line height
- Highlights — List of HighlightTile widgets with colored numbered circles and alternating accent colors
- Fixed Bottom Bar — White Container with shadow holding the price display and Book Now ElevatedButton

Screenshot

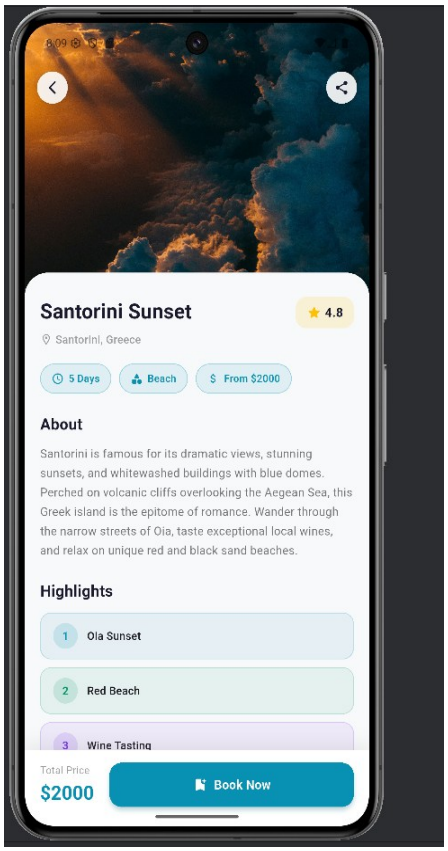

Figure 4 - Detail Screen (Bali Paradise)

2.3 Booking Screen

The Booking Screen provides a form-style UI for completing a travel booking. It is a StatefulWidget that tracks guest count, selected travel date, and form field values.

Layout Structure

The screen uses a Scaffold with a standard AppBar and a SingleChildScrollView body containing a Form widget with a GlobalKey for validation. All content scrolls vertically with consistent 20 px padding.

Key Components

- Trip Summary Card — Displays destination name, location, and duration inside a Card with a gradient icon placeholder
- Guest Details Form — Three validated TextFormField widgets for Full Name, Email Address, and Phone Number
- Travel Date Selector — DropdownButtonFormField with 5 hard-coded date options
- Guest Counter — Row with decrement/increment icon buttons controlling an integer state variable (1–10 guests)

- Price Summary Box — Styled Container showing per-person price, guest multiplier, and calculated total
- Confirm Button — Full-width ElevatedButton; on validation success, displays a success AlertDialog
- Confirmation Dialog — Modal AlertDialog with a green check icon, booking summary, and "Back to Home" button that pops two screens

Screenshots

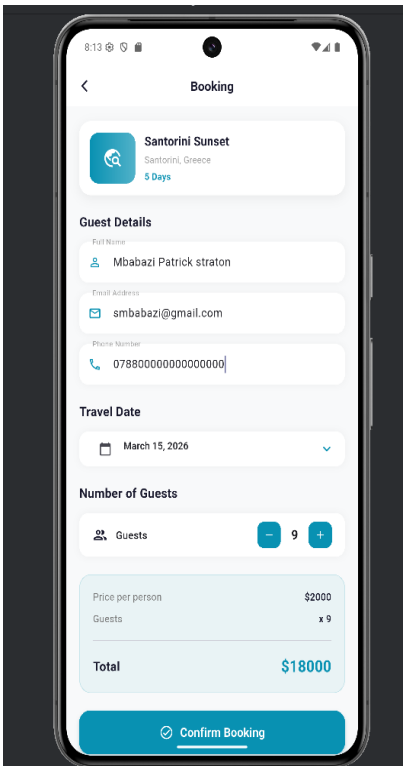

Figure 5 - Booking Screen (form view)

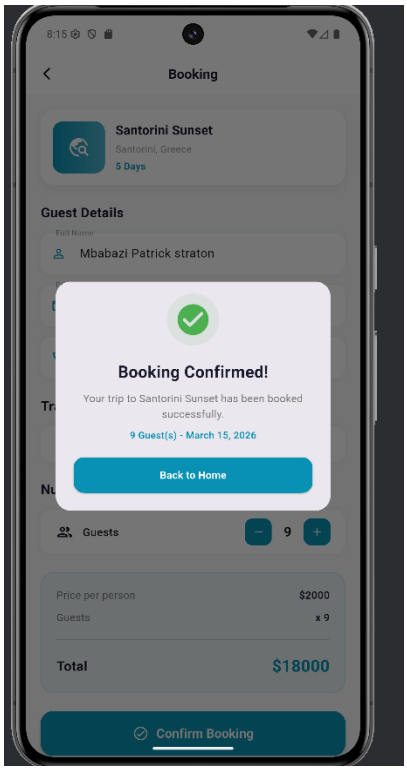

Figure 6 - Booking confirmed dialog

3. Navigation Flow

All navigation is implemented using Flutter's built-in Navigator stack with MaterialPageRoute. Data is passed between screens by constructing widgets with required parameters.

3.1 Navigation Diagram

3.2 Navigation Implementation

The following Navigator calls implement the full navigation flow:

Home → Detail

```
Navigator.push(context, MaterialPageRoute(builder: (_) => DetailScreen(destination: dest)));
```

The Destination object is passed directly as a constructor parameter to DetailScreen.

Detail → Booking

```
Navigator.push(context, MaterialPageRoute(builder: (_) => BookingScreen(destination: destination)));
```

The same Destination object is forwarded to BookingScreen for summary display and price calculations.

Booking → Home (after confirmation)

```
Navigator.pop(context); // dismiss dialog Navigator.pop(context); // back to Detail Navigator.pop(context); // back to Home
```

Three consecutive Navigator.pop() calls navigate the user back to the Home Screen after a successful booking confirmation.

4. Flutter Widgets Used

The application uses more than 18 distinct Flutter widgets. The table below lists each widget, where it appears, and its role in the UI.

Widget Name	Screen(s) Used	Purpose / Usage
MaterialApp	main.dart	Root widget; configures theme, font, and home route
Scaffold	All screens	Base screen structure with body, AppBar, and background
AppBar	Booking Screen	Top navigation bar with title and back button
SafeArea	Home Screen	Prevents content from overlapping system status bar
CustomScrollView	Home Screen	Unified scroll view combining Slivers for complex layout
SliverToBoxAdapter	Home Screen	Wraps non-sliver widgets (header, search, etc.) for use in CustomScrollView
SliverGrid	Home Screen	Renders destination cards in a 2-column responsive grid
ListView.builder	Home Screen	Horizontal scrolling category chip list
GridView	Home Screen	Underlying mechanism of SliverGrid for destination display
Stack	Detail Screen, Cards	Layers widgets for image overlays, gradient, and positioned elements
Positioned	Detail Screen, Cards	Anchors overlay widgets within a Stack (buttons, text, gradients)
SingleChildScrollView	Detail & Booking	Enables vertical scrolling for long-form content
Form	Booking Screen	Manages form state and enables field validation
TextFormField	Booking Screen	Validated text input fields (name, email, phone)
DropdownButtonFormField	Booking Screen	Travel date selection dropdown with hard-coded options
ElevatedButton	All screens	Primary action buttons (Book Now, Confirm Booking)
GestureDetector	Home, Cards	Tap handlers for category chips and destination cards
Container	All screens	Decoration, spacing, and shape for UI elements
Row / Column	All screens	Horizontal and vertical layout of children
Wrap	Detail Screen	InfoChip layout that wraps to next line on overflow
TextField	Home Screen	Live search input field
Icon / IconButton	All screens	Navigation and action icons throughout the app
Image.asset	Detail Screen, Cards	Displays destination images from local assets

ClipRRect	Destination Card	Clips card content to rounded rectangle corners
AlertDialog	Booking Screen	Booking confirmation success modal dialog
BoxDecoration	All screens	Gradients, shadows, borders on Container widgets
LinearGradient	Cards, Booking Card	Color gradients on image overlays and accent containers
BoxShadow	All screens	Depth shadows on cards, buttons, and search bar
SizedBox	All screens	Fixed spacing between layout elements
Padding	All screens	Consistent inner spacing around content areas
Text / TextRun	All screens	All text content with styled TextStyle
CircleAvatar / Circle	Home Screen	User avatar with gradient background
Divider	Booking Screen	Horizontal separator in price summary section

5. Custom Reusable Widgets

Four custom widget classes were created to improve code reuse, readability, and separation of concerns.

5.1 DestinationCard

File	lib/widgets/destination_card.dart
Type	StatefulWidget (manages favorite toggle state)
Parameters	destination: Destination, onTap: VoidCallback

The DestinationCard widget renders a destination as a full-bleed image card with a gradient overlay at the bottom. It contains the destination title, location, rating stars, price, and a toggleable favorite heart icon. It also shows a category label badge in the top-left corner. The card uses Stack + Positioned for all overlay elements, ClipRRect for rounded corners, and BoxShadow for depth.

5.2 InfoChip

File	lib/widgets/info_chip.dart
Type	StatelessWidget
Parameters	icon: IconData, label: String, color: Color

InfoChip renders a small pill-shaped tag combining an icon and a label. It uses the category accent color for both the icon and text, with a low-opacity fill and a matching border. Three InfoChip widgets appear in the Detail Screen showing trip duration, category, and starting price.

5.3 HighlightTile

File	lib/widgets/info_chip.dart
Type	StatelessWidget
Parameters	text: String, index: int

HighlightTile renders a single item in the destination's highlights list. Each tile has a numbered circular badge (using the index) with a rotating color palette, and a description label. It uses a lightly colored background and a subtle border matching the badge color.

5.4 HighlightTile & Helper Methods in DetailScreen

The DetailScreen also contains two private helper methods (_circleButton and _getCategoryColor/_getCategoryIcon) that act as micro-widget factories, keeping the build method clean and enabling easy reuse of the circular icon button pattern found in the header.

6. Layout Design Choices

6.1 CustomScrollView + Slivers (Home Screen)

The Home Screen uses CustomScrollView with SliverToBoxAdapter and SliverGrid instead of a simple Column inside a ListView. This allows the header, search bar, and category chips to scroll away together with the grid, creating a unified full-screen scroll experience. It also avoids the rendering constraints of nesting a scrollable inside another scrollable.

6.2 Stack + Positioned (Detail Screen)

The "Book Now" button bar is positioned at the bottom of the screen using a Positioned widget inside a Stack that wraps the entire screen. This keeps the CTA visible at all times while the rest of the content scrolls freely inside the SingleChildScrollView. A large bottom padding inside the scroll view ensures content is not hidden behind the bar.

6.3 Category-Driven Color Theming

Each destination category (Beach, Mountain, City, Adventure, Cultural) maps to a distinct accent color. This color is applied consistently across the category chip, the InfoChips, the "Book Now" button, the Booking Screen accent, and the price display. The mapping is implemented with a switch statement in both DetailScreen and BookingScreen, making the theme feel dynamic and destination-aware.

6.4 Image Error Handling

DestinationCard includes an errorBuilder on Image.asset. If an asset image fails to load, the card displays a colored gradient placeholder with the category icon centered. This makes the UI resilient during development when image assets may not be present.

6.5 Form Layout (Booking Screen)

Each TextFormField is wrapped in its own styled Container rather than relying on InputDecoration borders alone. This gives each field a card-like appearance with a white background and a soft box shadow, consistent with the visual language of the rest of the app.

7. Hard-Coded Data Structure

All destination data is defined in lib/data/travel_data.dart. No external API, database, or network call is used anywhere in the app.

7.1 Destination Model

```
class Destination {  final String id;  final String title;  final String location;  final String description;  final double price;  final double rating;  final String imagePath;  final String category;  final List<String> highlights; final int duration; }
```

7.2 TravelData Class

The TravelData class holds a static list of 8 Destination objects and two helper methods:

- getByCategory(String category) — Returns all destinations, or filters by the given category string
- getById(String id) — Returns the first destination with a matching id, or null

7.3 Destinations Included

Bali Paradise	Beach · Indonesia · $1,200 · ★4.8 · 7 days
Swiss Alps	Mountain · Switzerland · $2,500 · ★4.9 · 5 days
Tokyo Explorer	City · Japan · $1,800 · ★4.7 · 6 days
Machu Picchu Trek	Adventure · Peru · $1,500 · ★4.9 · 4 days
Santorini Sunset	Beach · Greece · $2,000 · ★4.8 · 5 days
Marrakech Medina	Cultural · Morocco · $900 · ★4.5 · 4 days
Patagonia Wild	Adventure · Chile · $2,200 · ★4.7 · 8 days
Kyoto Temples	Cultural · Japan · $1,600 · ★4.8 · 5 days

8. Lab Requirements Checklist

Widget Name	Screen(s) Used	Purpose / Usage
Requirement	✗ Missing	Implementation Details
18+ different Flutter widgets	✓ Met	Table in Section 4 lists 33 distinct widgets used
Custom reusable widget classes	✓ Met	DestinationCard, InfoChip, HighlightTile (Section 5)
Gradients or shadows	✓ Met	LinearGradient on cards; BoxShadow on all cards and buttons
Images from assets	✓ Met	Image.asset in DetailScreen and DestinationCard
Consistent color theme	✓ Met	Category accent colors + Indigo primary throughout
Responsive spacing	✓ Met	SizedBox, Padding, EdgeInsets used consistently
GridView or ListView.builder	✓ Met	SliverGrid + SliverChildBuilderDelegate on Home Screen
Card, Stack, Positioned, Row/Column	✓ Met	All used extensively across screens
Container with decoration	✓ Met	Used on all screens with BoxDecoration
Navigator.push and Navigator.pop	✓ Met	Full three-screen navigation flow implemented
Data passed between screens	✓ Met	Destination object passed Home→Detail→Booking
Home → Detail → Booking → Back flow	✓ Met	Three-screen stack navigation with triple pop
SingleChildScrollView	✓ Met	Used on Detail and Booking screens
Book Now button → Booking Screen	✓ Met	ElevatedButton in DetailScreen navigates to BookingScreen
Success dialog or snackbar	✓ Met	AlertDialog with confirmation shown on booking
Hard-coded data file	✓ Met	lib/data/travel_data.dart with full Destination model
Organized folder structure	✓ Met	data/, Screens/, widgets/, assets/ separation

9. Conclusion

The Wanderlust Travel App successfully meets all requirements specified in Lab 3b. The application demonstrates the following Flutter competencies:

- Advanced multi-screen layout composition using Slivers, Stacks, and ScrollViews
- Clean widget decomposition with reusable, parameterized custom widgets
- Navigator-based screen-to-screen navigation with data passing via constructors
- Dynamic UI theming driven by hard-coded data properties (category colors)
- Form construction and validation using Flutter Form and TextFormField
- Interactive UI elements including toggleable favorites, dropdown selectors, and animated counters
- Consistent Material 3 design language with gradients, shadows, and a coherent color palette

The project is structured for maintainability: adding a new destination requires only a new entry in TravelData.destinations — the UI adapts automatically. The separation of data, screens, and widgets into distinct folders lays a solid foundation for scaling the project to include real API calls and state management in future labs.

— End of Report —